

```

;;;;;;;;;;;;;
;; ECONOMIC COMPLEXITY E-COMMERCE MODEL
;;;;;;;;;;;;;

```

```
globals [
```

```

    capability-universe
    common-capabilities
    rare-capabilities
    product-templates

    capability-adjacency
    capability-requirements

    initial-sellers
    destroyed-links

    loan-size
    debt-threshold

    sellers-created
    entry-window
    entry-window-duration
    total-exits

```

```

]

breed [sellers seller]
breed [products product]
breed [buyers buyer]
undirected-link-breed [offers offer]

```

```

;;;;;;;;;;;;;
;; SELLERS
;;;;;;;;;;;;;

```

```
sellers-own [
```

```

    capability-set
    revenue
    profit
    reputation
    complexity
    diversification
    kc1
    cash
    debt
    risk-tolerance
    swarm-triggered?
    frontier-size
    eligible-frontier-size

```

```
]
```

```

;;;;;;;;;;;;;
;; BUYERS
;;;;;;;;;;;;;

```

```
buyers-own [
```

```

    budget
    preferences

```

```
]
```

```

;;;;;;;;;;;;;
;; PRODUCTS
;;;;;;;;;;;;;

```

```
products-own [
  product-name
  required-capabilities
  product-category
  price
  quality
  complexity
  sales
  ubiquity
  kp1
]
```

```
;;;;;;;;;;;;;
;; SETUP
;;;;;;;;;;;;;
```

```
to setup
```

```
  clear-all
```

```
  set sellers-created 0
  set destroyed-links 0
  set total-exits 0
  set loan-size 500
  set debt-threshold 2000
  set entry-window false
  set entry-window-duration 0
```

```
  set product-templates
  [
    ["BasicCommerce"
     ["marketing" "payments"]]

    ["Fulfillment"
     ["logistics" "warehousing"]]

    ["Marketplace"
     ["marketing"
      "payments"
      "supplier-network"]]

    ["CloudService"
     ["cloud-computing"
      "data-analytics"]]

    ["AIService"
     ["cloud-computing"
      "data-analytics"
      "ai-recommendation"]]
  ]
```

```
  set common-capabilities
  [
    "logistics"
    "marketing"
    "payments"
    "customer-service"
    "supplier-network"
    "warehousing"
    "last-mile-delivery"
  ]
```

```

    set rare-capabilities
[
  "data-analytics"
  "cloud-computing"
  "ai-recommendation"
]

    set capability-universe
[
  "logistics"
  "marketing"
  "payments"
  "customer-service"
  "data-analytics"
  "cloud-computing"
  "ai-recommendation"
  "supplier-network"
  "warehousing"
  "last-mile-delivery"
]

    set capability-adjacency
[
  ["marketing"
    ["payments"
      "customer-service"
      "supplier-network"]]

  ["payments"
    ["marketing"
      "customer-service"
      "supplier-network"]]

  ["customer-service"
    ["marketing"
      "payments"
      "data-analytics"]]

  ["supplier-network"
    ["marketing"
      "payments"
      "warehousing"]]

  ["warehousing"
    ["supplier-network"
      "logistics"
      "last-mile-delivery"]]

  ["logistics"
    ["warehousing"
      "last-mile-delivery"]]

  ["last-mile-delivery"
    ["logistics"
      "warehousing"]]

  ["data-analytics"
    ["customer-service"
      "cloud-computing"]]

  ["cloud-computing"
    ["data-analytics"
      "ai-recommendation"]]

  ["ai-recommendation"
    ["cloud-computing"]]
]

```

```

    set capability-requirements
  [
    ["marketing" []]

    ["payments"
     ["marketing"]]

    ["customer-service"
     ["marketing"]]

    ["supplier-network"
     ["marketing"
      "payments"]]

    ["warehousing"
     ["supplier-network"]]

    ["logistics"
     ["warehousing"]]

    ["last-mile-delivery"
     ["logistics"]]

    ["data-analytics"
     ["customer-service"]]

    ["cloud-computing"
     ["data-analytics"]]

    ["ai-recommendation"
     ["cloud-computing"
      "data-analytics"]]
  ]

  create-marketplace

  reset-ticks

end

```

```

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; CREATE MARKETPLACE
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;

```

```

to create-marketplace

  create-sellers Number-of-Sellers
  [
    initialize-seller
  ]

  set initial-sellers count sellers

  create-buyers Number-of-Customers
  [
    initialize-buyer
  ]

  create-global-products

  connect-sellers-to-products

  update-network-metrics

```

```

    update-reflections
end

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; PAY OPERATING COSTS
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;

to pay-operating-costs
    ask sellers
    [
        set cash cash - operating-cost
    ]
end

to add-capability-with-prerequisites [cap]
    let prereqs
        capability-prerequisites cap

    foreach prereqs
    [
        p ->

        if not member? p capability-set
        [
            add-capability-with-prerequisites p
        ]
    ]

    if not member? cap capability-set
    [
        set capability-set
            lput cap capability-set
    ]
end

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; INITIALIZE SELLER
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;

to initialize-seller

    set cash 1000
    set debt 0
    set risk-tolerance random-float 1
    set frontier-size 0
    set eligible-frontier-size 0

    set shape "person"
    set color blue
    set size 1.5
    set swarm-triggered? false

    setxy random-xcor random-ycor

    ;; capacidades iniciales sólo desde raíces

    set capability-set
        root-capabilities

    set capability-set
        remove-duplicates capability-set

```

```

set revenue 0
set profit 0

set reputation random-float 1

set complexity length capability-set

set diversification 0
set kc1 0

end

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; INITIALIZE BUYER
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;

to initialize-buyer

  set shape "person"
  set color red
  set size 1

  setxy random-xcor random-ycor

  set budget 100 + random 1000

  let n-preferences 1 + random 3

  set preferences
    n-of n-preferences capability-universe

end

; CREATE GLOBAL PRODUCTS

to create-global-products

  create-products Number-of-Products
  [

    set product-name
      word "Product-" who

    set shape "box"
    set color green
    set size 1.5

    setxy random-xcor random-ycor

    let template
      one-of product-templates

  set required-capabilities
    item 1 template

    set product-category
      one-of required-capabilities

    set complexity
      length required-capabilities

    set quality
      complexity + 1
  ]
end

```

```

    set price
      50 + complexity * 25

    set sales 0

    set ubiquity 0

    set kp1 0

  ]
end

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; PURCHASE PROCEDURE
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;

to make-purchase [chosen-product]

  let p [price] of chosen-product

  set budget budget - p

  ask chosen-product
  [
    set sales sales + 1
  ]

  let seller-agent
    one-of [link-neighbors] of chosen-product

  if seller-agent != nobody
  [
    ask seller-agent
    [
      set revenue revenue + p
      set profit profit + (0.30 * p)
      set cash cash + (0.30 * p)
    ]
  ]
end

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; ENTREPRENEURIAL ENTRY
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;

to entrepreneurial-entry

  if entry-window
  [

    let successful-sellers
      sellers with
      [
        cash > entry-threshold
      ]

    if any? successful-sellers
    [

      ;; entrada probabilística durante la ventana

      if random-float 1 < 0.10
      [

        let role-model

```

```

        max-one-of successful-sellers
        [cash]

create-sellers 1
[
    set cash 1000
    set debt loan-size

    set risk-tolerance random-float 1

    set shape "person"
    set color violet
    set size 1.5
    set swarm-triggered? false

    setxy random-xcor random-ycor

    set capability-set
        [capability-set] of role-model

    set revenue 0
    set profit 0

    set reputation random-float 1

    set complexity length capability-set

    set diversification 0
    set kc1 0

]

set sellers-created sellers-created + 1
connect-sellers-to-products

]
]
end

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; EVALUATE FINANCING
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;

to evaluate-financing
    ask sellers with [cash < 0]
    [
        if random-float 1 < risk-tolerance
        [
            set debt debt + loan-size
            set cash cash + loan-size
        ]
    ]
end

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; BUYING PROCESS
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;

to market-step

```



```

ask buyers
[
    let affordable-products
        products with
        [
            price <= [budget] of myself
        ]

    if any? affordable-products
    [
        let preferred-products
            affordable-products with
            [
                member? product-category [preferences] of myself
            ]

        ;; compra preferida

        if any? preferred-products
        [
            let chosen-product
                max-one-of preferred-products
                [
                    quality
                ]

            make-purchase chosen-product
        ]

        ;; si no encuentra preferencias

        if not any? preferred-products
        [
            let chosen-product
                one-of affordable-products

            make-purchase chosen-product
        ]
    ]
]
end

```

```

; Vendor-Product Connection
to connect-sellers-to-products

```

```

ask sellers
[
    let my-capabilities capability-set

    ask products
    [
        if is-subset? required-capabilities my-capabilities
        [
            create-offer-with myself
            [
                set color gray
            ]
        ]
    ]
]

```

```

    ]
  ]
]
end

; SELLER EXIT
to seller-exit

  ask sellers with
  [
    cash < 0
    and
    debt > debt-threshold
  ]
  [

    set destroyed-links
      destroyed-links + count my-offers

    set total-exits total-exits + 1

    die

  ]
end

;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
;; INNOVATION
;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;

to innovate

  ask sellers
  [

    let my-capabilities capability-set

    ask products
    [

      if not link-neighbor? myself
      [

        if is-subset? required-capabilities my-capabilities
        [

          create-offer-with myself
          [
            set color yellow
          ]

        ]

      ]

    ]

  ]

end

```

```

;;;;;;;;;;;;;
;; UPDATE VISUALIZATION
;;;;;;;;;;;;;

```

```
to update-colors
```

```

ask sellers
[
  set color scale-color blue complexity 1 10
]

ask products
[
  set color scale-color green complexity 1 10
]

```

```
end
```

```

; UPDATE NETWORK METRICS
to update-network-metrics

```

```

ask sellers
[
  set diversification count my-offers
]

ask products
[
  set ubiquity count my-offers
]

```

```
end
```

```
to evaluate-entry-window
```

```

if not entry-window
[
  let trigger-firm
    one-of sellers with
    [
      cash > entry-threshold
      and
      not swarm-triggered?
    ]

  if trigger-firm != nobody
  [
    ask trigger-firm
    [
      set swarm-triggered? true
    ]

    set entry-window true

    set entry-window-duration 25
  ]
]

```

```
end
```

```
; UPDATE ENTRY-WINDOW
```

```
to update-entry-window
```

```

if entry-window
[
    set entry-window-duration
    entry-window-duration - 1

    if entry-window-duration <= 0
    [
        set entry-window false
    ]
]

end

; UPDATE REFLECTIONS
to update-reflections

ask sellers
[
    ifelse any? my-offers
    [
        set kc1
        mean [ubiquity]
        of link-neighbors
    ]
    [
        set kc1 0
    ]
]

ask products
[
    ifelse any? my-offers
    [
        set kp1
        mean [diversification]
        of link-neighbors
    ]
    [
        set kp1 0
    ]
]

end

; LEARN CAPABILITIES
to learn-capabilities

ask sellers
[
    let probability-learning
    0.01 + (profit / 10000)

    if probability-learning > 1
    [
        set probability-learning 1
    ]

    if random-float 1 < probability-learning
    [

```

```

    let frontier []

    foreach capability-set
    [
        cap ->

        set frontier
            sentence frontier
            (capability-neighbors cap)

    ]

    set frontier
        remove-duplicates frontier

    set frontier
        filter
            [x -> not member? x capability-set]
        frontier
    set frontier-size
        length frontier

    let eligible-frontier []
    set eligible-frontier-size
    length eligible-frontier

foreach frontier
[
    candidate ->

    let prereqs
        capability-prerequisites candidate

    if is-subset? prereqs capability-set
    [
        set eligible-frontier
            lput candidate eligible-frontier
    ]
]

if not empty? eligible-frontier
[
    let new-capability
        one-of eligible-frontier

    set cash cash - learning-cost

    set capability-set
        lput new-capability capability-set
]

    ]

    set complexity length capability-set

]

end

to-report capability-neighbors [cap]

    let record
        filter
            [x -> item 0 x = cap]
        capability-adjacency

```

```

    ifelse empty? record
    [
        report []
    ]
    [
        report item 1 first record
    ]
end

to-report is-subset? [small-set big-set]

    report empty?
        filter
        [ x -> not member? x big-set ]
        small-set
end

to-report survival-rate

    report count sellers / initial-sellers
end

to-report mean-cash

    ifelse any? sellers
    [
        report mean [cash] of sellers
    ]
    [
        report 0
    ]
end

to-report mean-debt

    ifelse any? sellers
    [
        report mean [debt] of sellers
    ]
    [
        report 0
    ]
end

to-report mean-buyer-budget

    report mean [budget] of buyers
end

to-report exited-sellers

    report total-exits
end

to-report link-loss

    report destroyed-links
end

to-report mean-complexity

```

```

    ifelse any? sellers
    [
      report mean [complexity] of sellers
    ]
    [
      report 0
    ]
  end

to replenish-buyer-budgets

  ask buyers
  [
    set budget budget + buyer-income
  ]

end

to-report entrants

  report sellers-created

end

to-report seller-population

  report count sellers

end

to-report capability-prerequisites [cap]

  let record
    filter
      [x -> item 0 x = cap]
      capability-requirements

  ifelse empty? record
  [
    report []
  ]
  [
    report item 1 first record
  ]

end

to-report root-capabilities

  report
    map first
    filter
      [x -> empty? item 1 x]
      capability-requirements

end

to-report net-population-change

  report count sellers - initial-sellers

end

to-report customer-service-sellers

```

```

    report count sellers
      with [member? "customer-service"
               capability-set]
end
to-report data-analytics-sellers

  report count sellers
    with [member? "data-analytics"
             capability-set]
end
to-report cloud-sellers

  report count sellers
    with [member? "cloud-computing"
             capability-set]
end
to-report ai-sellers

  report count sellers
    with [member? "ai-recommendation"
             capability-set]
end

to-report zero-offer-products

  report count products
    with [ubiquity = 0]
end

to-report zero-offer-sellers

  report count sellers
    with [diversification = 0]
end

to-report mean-frontier-size

  ifelse any? sellers
  [
    report mean [frontier-size] of sellers
  ]
  [
    report 0
  ]
end

to-report mean-eligible-frontier-size

  ifelse any? sellers
  [
    report mean [eligible-frontier-size] of sellers
  ]
  [
    report 0
  ]
end

;;;;;;;;;;;;;;
;; G0

```


;;;;;;;;;;;;;

```
to go
  replenish-buyer-budgets
  pay-operating-costs
  market-step
  learn-capabilities
  innovate
  evaluate-entry-window
  entrepreneurial-entry
  update-entry-window
  evaluate-financing
  seller-exit
  update-network-metrics
  update-reflections
  update-colors
  tick
end
```